

HyperRPS: An Efficient Multi-Fidelity Hyper-Parameter Tuning Framework for Rock Particle Segmentation

Yili Ren[†], Beicheng Xu[†], Huaijun Jiang, Xi Liu, Ping Zhang, Qian Liang, Yu Shen, and Bin Cui*

Abstract: Rock particle segmentation is essential for identifying thin sections accurately, yet tuning deep learning models for this task is challenging. Existing studies have attempted auto-tuning techniques but face three limitations, including limited search space, inefficient search, and high demand for storage space. In this paper, we introduce HyperRPS, a novel multi-fidelity hyper-parameter tuning framework tailored for rock particle segmentation, which addresses these issues simultaneously. First, by augmenting the search space with diverse hyper-parameters, we unleash the potential of the segmentation model. Second, to accelerate the hyper-parameter search, we propose a tuning algorithm leveraging multi-fidelity surrogate fitting and time constraint modeling. Third, we develop a storage management technique to reduce the space requirements for asynchronous scheduling. Experimental results on both public and rock particle segmentation datasets demonstrate HyperRPS's superior performance. Specifically, on the rock particle segmentation dataset, HyperRPS achieves notable improvements of 8.26% and 2.78% on average precision when tuning Mask2Former (Swin-B) and Mask2Former (Swin-L), respectively, outperforming existing state-of-the-art methods.

Key words: rock particle segmentation; hyper-parameter tuning; multi-fidelity optimization; Bayesian optimization (BO)

1 Introduction

Rock particle segmentation^[1, 2] is the foundation of

mineral component analysis and rock structure identification. It is also the core content of intelligent thin-section identification. The segmentation of rock

- Yili Ren is with the Research Institute of Petroleum Exploration & Development, PetroChina, Beijing 100083, China, also with the Artificial Intelligence Technology R&D Center of Exploration and Development, China National Petroleum Corporation, Beijing 100083, China, also with the National Key Laboratory for Multi-Resources Collaborative Green Production of Continental Shale Oil, Daqing 163000, China, and also with Department of Automation, Tsinghua University, Beijing 100084, China. E-mail: renyili@petrochina.com.cn.
- Beicheng Xu, Huaijun Jiang, Yu Shen, and Bin Cui are with the Key Laboratory of High Confidence Software Technologies (MOE), School of Computer Science, Peking University, Beijing 100871, China. E-mail: beichengxu@stu.pku.edu.cn; jianghuaijun@pku.edu.cn; shenyu@pku.edu.cn; bin.cui@pku.edu.cn.
- Xi Liu is with Research Institute of Petroleum Exploration & Development, PetroChina, Beijing 100083, China, and also with Artificial Intelligence Technology R&D Center of Exploration and Development, China National Petroleum Corporation, Beijing 100083, China. E-mail: liuxi3@petrochina.com.cn.
- Ping Zhang and Qian Liang are with Digital and Intelligent Management Department, PetroChina Changqing Oilfield Company, Shaanxi 710018, China. E-mail: zhangp46_cq@petrochina.com.cn; liangq_cq@petrochina.com.cn.

[†] Yili Ren and Beicheng Xu contribute equally to this paper.

* To whom correspondence should be addressed.

Manuscript received: 2024-09-06; revised: 2025-05-07; accepted: 2025-05-07

particles allows for the simultaneous identification of particle edges and mineral attribute information. Based on the results of rock particle segmentation, it is possible to achieve a precise and quantitative calculation of the content proportion of each mineral type. Rock particle segmentation can quantitatively characterize the structural features of rocks and provide quantifiable basic data for the evaluation of diagenesis. The evaluation of particle size, sorting, and roundness depends on the size and shape of rigid particles, contact mode depends on the position of terrigenous clasts, and the proportion of interstitial materials directly influences the cementation type.

Traditional rock particle segmentation relies on superpixel analysis^[3–5], watershed^[6, 7], morphological processing^[8], or other algorithms^[9]. These algorithms require complex parameter adjustment processes. Meanwhile, the model is relatively shallow and has poor robustness and generalization ability to samples from different blocks. Intelligent particle segmentation based on deep learning technology^[10, 11] is the trend of core analysis. This approach relies on data-driven feature extraction. By learning from the rich features of a large number of samples, the deep learning model can obtain deep and dataset-specific feature representation. This results in feature extraction with more robustness and rock particle segmentation with improved efficiency, accuracy, and generalization ability.

Recently, the deep segmentation networks, including Mask R-CNN^[12] and Mask2Former^[13], have achieved state-of-the-art performance in rock particle segmentation. However, the effectiveness of deep learning models hinges heavily on the choice of their hyper-parameters, such as the learning rate. Manually tuning the hyper-parameters requires specialized expertise and can be arduous, particularly when dealing with large hyper-parameter search spaces. This issue is further underscored in the rock particle segmentation task, where the need for hyper-parameter tuning is exceptionally pressing. Since hyper-parameters are typically selected or tuned for training models on standard segmentation datasets, tuning them for domain-specific datasets can lead to more significant performance gains. In our experiments, we observe a maximum improvement of 2.69 average precision (AP) (+8.26%) on the rock particle segmentation dataset, compared with 0.79 AP (+1.36%) on a standard vision

dataset.

A variety of automatic hyper-parameter tuning (HPT) techniques have been proposed^[14], including random search^[15], evolutionary algorithms^[16, 17], Bayesian optimization (BO)^[18–20], etc. However, due to the ever-growing cost of model training, traditional HPT methods are no longer practical, requiring dozens or even hundreds of model evaluations, each taking several hours. To address this challenge, multi-fidelity algorithms^[21–24] incorporate an early stopping mechanism to evaluate models with lower budgets. For instance, a neural network with a particular hyper-parameter configuration is first trained for a few epochs to check if it performs well. If the network ranks among the top 1/3, it will be given additional resources and trained for more epochs. Otherwise, its evaluation will stop early, and a new hyper-parameter configuration will be proposed and evaluated.

The multi-fidelity algorithms enable the tuning of hyper-parameters in large neural networks. Despite this, there are still some limitations in existing HPT algorithms as follows:

L1: Limited search space. The search space for hyper-parameter tuning is frequently confined to standard scenarios rather than being intricately tailored to specific domains. Served as a fundamental step of hyper-parameter tuning, a poorly designed search space can hinder the effectiveness of the subsequent tuning algorithm, leading to sub-optimal outcomes. It is crucial to exercise caution and draw upon a domain expert's knowledge when selecting hyper-parameters within the search space.

L2: Inefficient search. Existing methods are ineffective as they either ignore previous history^[21, 23] or only consider observations with a complete evaluation budget^[22]. Since many observations are evaluated with a low budget (i.e., a few epochs), ignoring the rich information contained in these observations will slow the search process. Additionally, none of the existing multi-fidelity approaches consider the training time of a model, which can vary significantly due to different hyper-parameter configurations. The stragglers will take up plenty of the search time, reducing the search space's exploration.

L3: High demand for storage space. To resume model training with larger budgets, asynchronous multi-fidelity methods like ASHA^[23] store checkpoints

of neural networks. The uncontrolled storage process can fill the space and raise unexpected errors, which becomes an issue when tuning large models. For example, tuning Mask2Former (Swin-L) using ASHA takes up more than 100 GB of storage space for one trial. More importantly, most checkpoints remain unused until the end of the search.

Contributions. In this paper, we propose HyperRPS, a multi-fidelity hyper-parameter tuning framework, which can efficiently unearth the optimal hyper-parameters for Mask2Former on the rock particle segmentation task. Compared with existing approaches, HyperRPS addresses all the issues mentioned above, and the main contributions are summarized as follows: (1) HyperRPS elaborately designs a search space (L1) for the hyper-parameter tuning task. To further unleash the potential of the tuning algorithm, the search space is augmented with multiple types of hyper-parameters, including optimizer-related and data augmentation hyper-parameters. All hyper-parameters are tuned simultaneously. (2) HyperRPS proposes a multi-fidelity hyper-parameter tuning algorithm with asynchronous scheduling. To speed up the search process (L2), HyperRPS fits a multi-fidelity surrogate model using all history observations, including partially evaluated ones. A multi-fidelity time constraint is introduced to restrict the time-consuming candidates. HyperRPS proposes a dynamic checkpoint retention algorithm for storage management (L3), significantly reducing the storage space by over 75% without downgrading the performance. (3) Empirical results on both public and rock particle segmentation datasets demonstrate the state-of-the-art performance of HyperRPS. Concretely, HyperRPS outperforms other baselines by achieving an average improvement of 0.79 (+1.36%) and 2.69 (+8.26%) on AP when tuning Mask2Former (Swin-B) on the public and the rock particle segmentation dataset, respectively, and improves the AP of Mask2Former (Swin-L) by 1.16 (+2.78%) on the rock particle segmentation dataset.

The rest of this paper is organized as follows. Section 2 introduces the related works. Section 3 describes the rock particle segmentation dataset and the deep learning model that HyperRPS tunes. Section 4 describes the proposed hyper-parameter tuning framework HyperRPS in detail. Section 5 reports the experimental results by comparing our method with state-of-the-art methods. Section 6 provides a conclusion for this paper.

2 Related Work

The analysis of color variations and texture trends in images to extract rock particle edge is a common method in traditional digital core analysis technology^[3, 6, 25]. Fang et al.^[8] utilized algorithms including segmentation, filtering, and morphological processing to process thin-section images, thereby obtaining particle edges. Jiang et al.^[9] proposed a mineral component segmentation method based on DBSCAN^[26] and mathematical morphology, achieving the extraction of target rock particles. Considering the boundary distinction of mineral particles is not obvious and ambiguous in complex diagenetic environments, and determining mineral types needs to comprehensively observe plane-polarized light and interference color changes of multiple angles cross-polarized light, it is a development trend for scholars to figure out how to comprehensively utilize multiple light sources and angles of images, combined with interference color changes and other laws, to improve the accuracy of rock particle segmentation^[4, 5, 7, 27, 28]. Hu et al.^[29] achieved particle segmentation by comprehensively analyzing the similarity of superpixels under multi-angle polarization, achieving regional fusion. Zhou et al.^[30] analyzed coherent features based on angle-domain optical characteristics, spatial scale information, and anisotropy, achieving the extraction of particle edges. Although some achievements have been made in traditional particle segmentation, most of them extract shallow features of the image, without incorporating semantic information, which can easily lead to under-segmentation and over-segmentation.

Deep learning relies on data-driven methods, iteratively training networks on datasets to construct mappings between segmentation results and images. Deep neural networks can extract deep semantic information and more abstract pattern features. The Mask R-CNN^[12] network and watershed were utilized by Si et al.^[10] to achieve rock particle segmentation. The U-Net^[31] network was used by Pattnaik et al.^[11] to train and segment three types of carbonate rocks: Granular limestone, conglomerate limestone, and mudstone limestone.

However, the deep learning networks in existing approaches are built for general domain scenes (red, green, blue (RGB) three-channel images). There is limited research on applying deep learning to multiple

polarization images since these images require additional processing. Conversely, in order to achieve precise rock particle segmentation, we collect the segmentation dataset under plane-polarized light 0° and multiple cross-polarized lights. We also utilize a state-of-the-art segmentation model Mask2former^[13], better than Mask R-CNN, for the segmentation task.

In automated HPT^[32, 33], rudimentary strategies are Grid Search^[34] and Random Search^[15]. In recent years, BO^[18–20] has emerged as a sophisticated approach, gaining widespread adoption for HPT tasks. This technique involves the training of a surrogate model leveraging historical evaluation data alongside the employment of an acquisition function to balance the exploration and exploitation of the search space. Evolutionary algorithms (EAs)^[16, 17] introduce a model-free technique for HPT. These algorithms manage a hyper-parameter population, continuously evolving through selection, mutation, and crossover processes to generate viable new candidates.

Multi-fidelity approaches such as Successive Halving^[35] and HyperBand^[21] integrate an early stopping mechanism, permitting the assessment of hyperparameters under lower budgets. BOHB^[22] merges the principles of BO with HyperBand, facilitating an efficient exploration of the search space informed by previous data. Additionally, ASHA^[23] enhances the parallel efficiency inherent in the successive halving approach by incorporating an asynchronous scheduling mechanism.

3 Preliminary

In this section, we introduce the rock particle

segmentation dataset, the deep learning model for the segmentation task, and the automated hyper-parameter tuning approaches as preliminary.

3.1 Rock particle segmentation dataset

In this paper, we establish a segmentation dataset of rock particle images to facilitate automated and intelligent particle analysis and rock structure evaluation. Each sample in the dataset consists of seven thin-section images collected by an Axio Scan Z7 microscope from the same particle sample and the same location under plane-polarized light 0° and multiple cross-polarized lights. A sample is illustrated in Fig. 1.

We model the rock particle segmentation task as an instance segmentation problem. The segmentation dataset consists of 10 000 samples with particle edge information and category attributes. Each rock particle is assigned one of 19 labels. For fair model comparison, we randomly select 8000 samples as the training set, 1000 samples as the validation set, and 1000 samples as the test set.

3.2 Deep learning model for segmentation

To achieve precise particle segmentation, we select the state-of-the-art deep segmentation model Mask2Former^[13] for rock particle segmentation. Since each sample in the rock particle segmentation dataset contains seven images, we fuse the features of seven images as network input to extract fuzzy features.

The Mask2Former network is constructed of three components, including a backbone feature extractor, a pixel decoder, and a Transformer decoder. The backbone feature extractor is first applied to extract

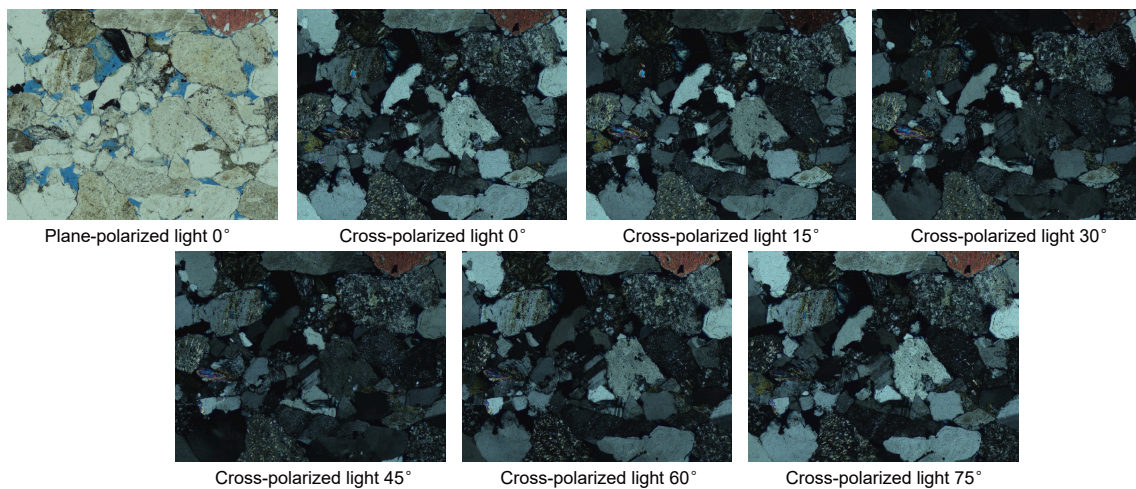


Fig. 1 A sample of the rock particle images under different light sources and angles.

deep features from input images. In this paper, we employ Swin-Transformer^[36] as the backbone feature extractor. Then, the pixel decoder, utilizing a feature pyramid structure, fuses the feature maps of different resolutions output by the backbone feature extractor from top to bottom, obtaining high-resolution embedding features for each pixel. The Transformer decoder operates on the multi-scale image features output by the pixel decoder to process object queries, extracts features for classification for each object, and classifies them through a multilayer perceptron (MLP) layer, as well as generates an embedding for multiplying with the high-dimensional features to obtain object segmentation results.

Finally, we perform post-processing on the segmentation results of each particle output from the network in seven images and merge them into one mask. The process of segmenting rock particle images is displayed in Fig. 2.

3.3 Automated hyper-parameter tuning

Multi-fidelity tuning. Given a performance metric (e.g., model accuracy), hyper-parameter tuning aims to find an optimal hyper-parameter configuration that achieves the best model performance. The formulation is as follows:

$$x^* = \arg \max_{x \in \mathcal{X}} P(M(x, \mathcal{D}_{\text{train}}), \mathcal{D}_{\text{valid}}) = \arg \max_{x \in \mathcal{X}} f(x) \quad (1)$$

where x^* denotes the optimal hyper-parameter configuration and $\mathcal{X}$ represents the search space. The model, M , is first trained with hyper-parameter x on the training dataset $\mathcal{D}_{\text{train}}$ and subsequently evaluated by the performance metric P on the validation dataset

$\mathcal{D}_{\text{valid}}$. The hyper-parameter evaluation process is simplified to a function $f: \mathbb{R}^d \rightarrow \mathbb{R}$, which maps the d -dimensional hyper-parameter configuration x to the metric score y . The other items are abstracted in this definition since they remain invariant in the tuning process.

In the realm of multi-fidelity tuning, the model is trained under a given resource budget r (e.g., trained for only a few epochs) for faster evaluation. Therefore, the evaluation function is reframed as

$$y = f(x, r) \quad (2)$$

When identifying the optimal solution x^* , a hyper-parameter configuration is selected only if evaluated with the maximum resource (denoted as R), as this typically yields higher and more robust results.

Equation (2) illustrates the quintessence of multi-fidelity tuning algorithms, which pivot on the decision of which hyper-parameter configuration x to evaluate and what resource r to allocate in each iteration.

BO. HyperRPS adopts the idea of BO^[18–20] for guided search space exploration. The core principle of BO involves modeling the hyper-parameter search space through observation data utilizing a machine learning model, as known as surrogate model, and balancing the space's exploration and exploitation via an acquisition function. The vanilla BO iterates over a cycle of suggestion and evaluation phases, encompassed within three distinct steps in each iteration: (1) A probabilistic surrogate model M_s is trained using the history data $D = \{(x_i, y_i)\}_{i=1}^{n-1}$, where x_i is an evaluated hyper-parameter configuration and y_i is the corresponding performance metric. n denotes the

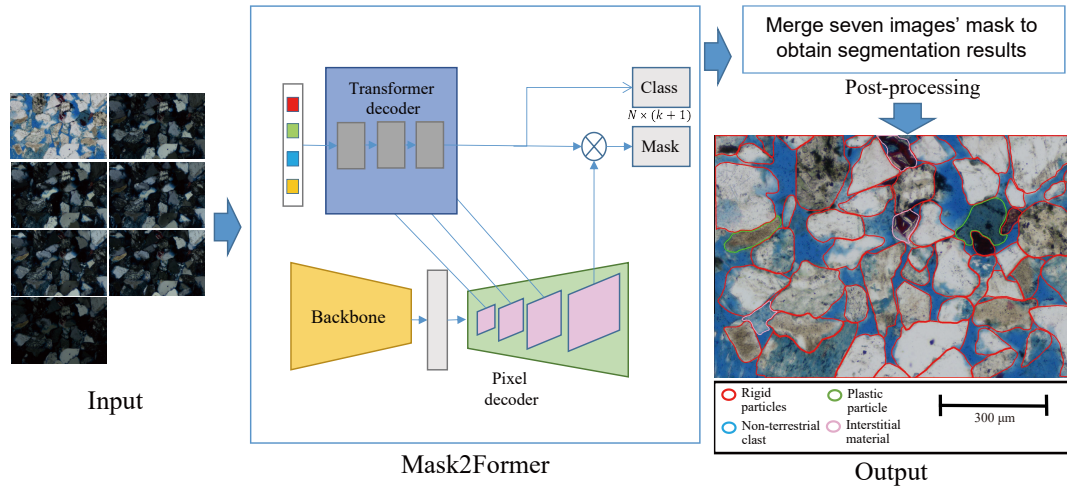


Fig. 2 Process of rock particle segmentation.

current number of iterations. (2) Upon the model's prediction, an acquisition function α calculates the evaluation value of new hyper-parameter candidates. The hyper-parameter configuration with the highest acquisition value is selected, $x^* = \arg \max_{x \in \mathcal{X}} \alpha(x, M_s)$. (3) The performance y_n of the selected hyper-parameter x_n is evaluated on the objective function f . Then the pair of result (x_n, y_n) is added to the tuning history D .

A common choice of the surrogate model is the Gaussian process (GP)^[37] due to its strong modeling ability with few data and closed-form inference. The predicted performance of the new hyper-parameter candidate follows a normal distribution as follows:

$$\begin{aligned} y &\sim \mathcal{N}(\mu(x), \sigma^2(x)), \\ \mu(x) &= \mathbf{K}(x, x')\mathbf{K}^{-1}(x', x')y', \\ \sigma^2(x) &= \mathbf{K}(x, x) - \mathbf{K}(x, x')\mathbf{K}^{-1}(x', x')\mathbf{K}(x', x) \end{aligned} \quad (3)$$

where x' and y' are observed tuning history and $\mathbf{K}$ denotes the covariance matrix. Here, $\mathcal{N}(\mu(x), \sigma^2(x))$ represents a univariate normal distribution with mean $\mu(x)$ and variance $\sigma^2(x)$, where $\mu(x)$ is the predicted mean performance of the candidate x , and $\sigma^2(x)$ is the corresponding predictive uncertainty

4 Methodology

In this section, we first give an overview of the proposed framework. Then, we introduce the search space design and describe the core multi-fidelity tuning algorithm. Finally, we provide the implementation details of the tuning framework.

4.1 Framework overview

Figure 3 illustrates the framework of HyperRPS. The optimization engineer inputs the designed search space (Section 4.2) and the time budget into the system. The multi-fidelity tuning algorithm of HyperRPS (Section 4.3) consists of three major components: (1) The optimizer suggests hyper-parameters based on evaluation history. It utilizes all history, including partially evaluated ones, to train a multi-fidelity surrogate to generate new hyper-parameter candidates. Additionally, a time-constraint model is fitted to avoid the time-consuming hyper-parameter configurations. (2) The asynchronous scheduler schedules individual evaluation tasks. It adopts an early stopping mechanism and decides on the resource budget (i.e., training epochs) for each task. If a previously stopped hyper-parameter configuration is selected to continue training, the scheduler tells the evaluator to load the

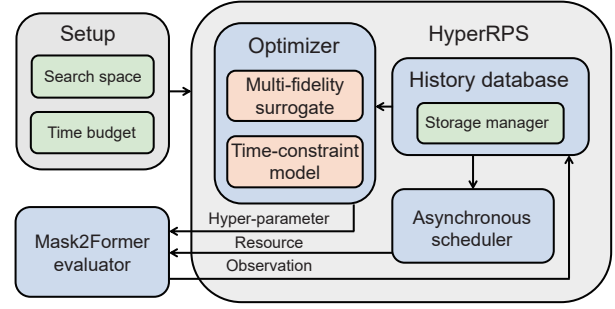


Fig. 3 Framework of HyperRPS.

neural network's checkpoint from the history database. (3) The history database records performance observation and resource budget of all hyper-parameters, guiding the optimizer modeling. It also stores the neural network's checkpoints for continuous training, with storage management to reduce the cost of storage space.

During optimization, HyperRPS interacts with the Mask2Former evaluator to continuously select the hyper-parameters and improve the performance of the neural network. Once an evaluator is free, the optimizer generates a hyper-parameter configuration, and the scheduler assigns a resource budget. The evaluator then trains a Mask2Former model under the given hyper-parameter and resource and reports the performance observation to the history database. The communication between HyperRPS and the evaluator employs a distributed message queue structure (Section 4.4). The optimization ends if the setup time budget is reached.

4.2 Search space design

The design of an adequate search space is a crucial step in the process of hyper-parameter optimization for machine learning models. It serves as the foundation upon which the optimization algorithms operate, determining the range and types of hyper-parameters to be explored for improving model performance. A well-designed search space can significantly enhance the efficiency and outcomes of the optimization process, leading to more robust and high-performing models.

In this context, we elaborately design a search space for tuning Mask2Former on the rock particle segmentation task that aims to cover a comprehensive spectrum of effective hyper-parameters, thereby maximizing the potential for performance improvements. This search space is uniquely augmented to include not only the commonly tuned hyper-parameters, such as learning rate and optimizer

type, but also extends to encompass a variety of other critical parameters. Specifically, as shown in Table 1, the search space incorporates multiple types of hyper-parameters:

(1) **Optimizer-related hyper-parameters.** These include traditional parameters like the optimizer (e.g., stochastic gradient descent (SGD) and adaptive moment estimation with decoupled weight decay (AdamW)), and the learning rate, but also extend to more specific ones such as weight decay^[38], momentum^[39, 40], and the backbone multiplier. The inclusion of optimizer-related hyper-parameters is crucial, as they directly influence the model’s convergence speed and final performance.

(2) **Data augmentation hyper-parameters.** Data preprocessing and augmentation show a significant impact on model performance. Therefore, we incorporate in the search space hyper-parameters for all commonly used image augmentation operations supported by the Detectron2^[41] library. This allows for an extensive exploration of data manipulation techniques for more generalized and robust models.

(3) **Clip gradient hyper-parameter.** To prevent the issue of exploding gradients, a clip gradient hyper-parameter is included. This parameter is essential for maintaining the stability of the model during training, especially in deep neural networks^[42].

The search space is a mix of both float and categorical hyper-parameters. Some parameters are tuned in the logarithmic scale if the order of magnitude is significant. For each hyperparameter, we construct a moderate range of values by varying around the default settings of Mask2Former^[13]. However, depending on different time budgets, the range can be further expanded or narrowed. By simultaneously tuning all hyper-parameters included in the search space, we aim to unleash the potential of the tuning algorithm fully.

This approach enables the discovery of optimal combinations of hyper-parameters that may not be apparent when tuning parameters in isolation, thereby maximizing model performance and efficiency.

4.3 Multi-fidelity tuning algorithm

The ensuing sub-sections delve into the proposed multi-fidelity tuning algorithm in HyperRPS, including the multi-fidelity surrogate approach, the time-constraint modeling technique, and the asynchronous scheduling mechanism with storage management. We present a summary of the tuning workflow at the end.

4.3.1 Multi-fidelity surrogate

The surrogate model in vanilla BO takes the hyper-parameter as input and outputs the predicted performance. However, observations are obtained using different resources in the multi-fidelity tuning scenario. On the one hand, since training with more resources typically yields higher performance, the results under different budgets are not aligned, making it challenging to train a single surrogate. On the other hand, the multi-fidelity data $D_{MF} = \{(x_i, y_i, r_i)\}_{i=1}^n$ contains richer information than the data only evaluated with the maximum resource. Discarding the multi-fidelity data leads to low efficiency of surrogate model fitting and slow convergence of hyper-parameter tuning.

To address this issue, HyperRPS proposes a multi-fidelity surrogate approach. At the surrogate training phase, the vector of the hyper-parameter is concatenated with its allocated resource budget as $x_{MF} = (x, r)$, where x represents the hyper-parameter vector used in the vanilla BO, r denotes the resource, and x_{MF} is the multi-fidelity input fed to the surrogate model. At the inference phase, the prediction for a new candidate is formulated as Eq. (4),

Table 1 Designed search space.

Category	Hyper-parameter	Type	Range
Optimizer	Optimizer type	Categorical	{SGD, AdamW}
	Learning rate	Float (log)	$[10^{-5}, 10^{-3}]$
	Backbone multiplier	Float (log)	$[0.01, 1.00]$
	Weight decay	Float (log)	$[10^{-5}, 0.5]$
	Momentum	Float	$[0.80, 0.99]$
DataAugmentation	Brightness	Categorical	{false, true}
	Contrast	Categorical	{false, true}
	Saturation	Categorical	{false, true}
Other	Clip gradients value	Float (log)	$[10^{-3}, 1.000]$

$$\begin{aligned}
y &\sim \mathcal{N}(\mu(x), \sigma^2(x)), \\
\mu(x) &= \mathbf{K}((x, R), x'_{\text{MF}}) \mathbf{K}^{-1}(x'_{\text{MF}}, x'_{\text{MF}}) y', \\
\sigma^2(x) &= \mathbf{K}((x, R), (x, R)) - \mathbf{K}((x, R), x'_{\text{MF}}) \cdot \\
&\quad \mathbf{K}^{-1}(x'_{\text{MF}}, x'_{\text{MF}}) \mathbf{K}(x'_{\text{MF}}, (x, R)) \quad (4)
\end{aligned}$$

The resource budget r is logarithmic-scaled to improve the stability of the Gaussian process model. The new hyper-parameter candidate is concatenated with the maximum resource R to make the prediction, representing the performance of the hyper-parameter with complete training.

4.3.2 Time-constraint modeling

The choice of hyper-parameters not only impacts a model's final performance, but also influences its training process. Specifically, the training time varies with different hyper-parameter settings. For instance, complex data augmentation can slow down a neural network's training. In traditional hyper-parameter tuning, evaluation time for a configuration is usually disregarded, and different algorithms are compared under the same number of iterations. However, by setting time as the search budget, controlling each trial's duration becomes crucial. Within the same budget, exploiting a time-consuming candidate can decrease the search space exploration and harm the tuning process.

To address this, HyperRPS develops a time-constraint modeling technique that integrates the multi-fidelity time constraint into the acquisition function, restricting the exploitation of time-consuming hyper-parameters. One challenge in the multi-fidelity scenario is that training time depends on the allocated resource budget, apart from the hyper-parameter. Nonetheless, the training time increases linearly with resources, such as training epochs, making the constraint function concise.

The base acquisition function in HyperRPS is the expected improvement (EI)^[43], representing the expected improvement of a candidate over the best observed, as shown in Eq. (5),

$$\alpha_{\text{EI}}(x) = E(\max(y - y_{\text{best}}, 0)) \quad (5)$$

where y_{best} is the best observed performance. When using the Gaussian process to predict y , the EI function can be expressed in closed form, as Eq. (6)

$$\alpha_{\text{EI}}(x) = (\mu(x) - y_{\text{best}}) \Phi(\gamma(x)) + \sigma(x) \phi(\gamma(x)), \quad (6)$$

where $\gamma(x) = \frac{\mu(x) - y_{\text{best}}}{\sigma(x)}$, and $\Phi(\cdot)$ and $\phi(\cdot)$ are the

standard normal cumulative distribution function and probability density function.

To accommodate constraints within the tuning process, HyperRPS integrates the expected improvement with constraints (EIC)^[44]. The acquisition function is calculated as Eq. (7) to introduce a training time constraint:

$$\alpha_{\text{EIC}}(x) = \alpha_{\text{EI}}(x) P(t < T_{\text{max}}) \quad (7)$$

where T_{max} is a pre-defined training time threshold and t is the training time of configuration x . The constrained acquisition function thus represents the product of the expected improvement and the likelihood of adhering to the specified time constraint, effectively penalizing candidate solutions deemed excessively time-consuming and thereby precluding their selection for evaluation.

We employ another Gaussian process to predict the training time of a particular hyper-parameter configuration. This time surrogate accepts the hyper-parameter vector as input and yields the probability distribution of the predicted time, described by Eq. (6),

$$t \sim \mathcal{N}(\mu_t(x), \sigma_t^2(x)) \quad (8)$$

The acquisition function under constraints can then be articulated through a closed-form equation as Eq. (9),

$$\alpha_{\text{EIC}}(x) = \alpha_{\text{EI}}(x) \Phi\left(\frac{T_{\text{max}} - \mu_t(x)}{\sigma_t(x)}\right) \quad (9)$$

In multi-fidelity scenarios, a single hyper-parameter configuration might correlate with multiple training time outcomes contingent upon the allocated resources. To accommodate this, we normalize the training time against the resources, as articulated by Eq. (10),

$$t = \frac{R}{r_x} t_x \quad (10)$$

where t_x and r_x represent the actual training duration and the corresponding resource for hyper-parameter x , with R denoting the maximum resource. This approach facilitates the utilization of a single model to encapsulate the multi-fidelity time constraints, thus enhancing the overall efficiency of the tuning process.

4.3.3 Asynchronous scheduling with storage management

In the architecture of HyperRPS, the asynchronous scheduler plays a pivotal role in determining the allocation of resources for various hyper-parameter configurations. The underlying principle is to evaluate

more candidates with fewer resources and give more resources to promising ones to finish their evaluation, thereby enriching the suggestion algorithm with an augmented dataset of observations.

HyperRPS employs an asynchronous multi-fidelity scheduling mechanism inspired by ASHA^[23]. The asynchronous nature of this mechanism ensures that an evaluator is immediately assigned a new evaluation task upon availability, circumventing the need for a synchronous barrier prevalent in methodologies such as Successive Halving^[35] and HyperBand^[21], thus enhancing the efficiency of the tuning process. Despite its advantages, ASHA encounters challenges, notably (1) the unguided nature of the search process and (2) the uncontrolled storage management, rendering it less efficacious for tuning large neural network models. To address these impediments, we advocate for the incorporation of a multi-fidelity surrogate for hyper-parameter suggestion and a dynamic checkpoint retention algorithm for storage management.

Given an initial resource r_{init} , a maximum resource R , and a promote proportion η (typically $\eta = 3$), the resource levels for evaluation are defined as Eq. (11),

$$\text{levels} = [r_{\text{init}}, \eta r_{\text{init}}, \dots, \eta^{K-1} r_{\text{init}}, \eta^K r_{\text{init}}] \quad (11)$$

where K refers to the number of early-stopping levels, and $\eta^K r_{\text{init}} = R$ represents the complete level. An evaluator with a newly suggested hyper-parameter is first allocated with the initial resource r_{init} . Upon reaching a predefined resource level, as delineated in Eq. (11), the neural network’s training is halted, and the evaluation result, along with the model checkpoint, is stored in the history database. Once an evaluator is free, the asynchronous scheduler first ascertains the presence of a promising hyper-parameter candidate within the history database. The hyper-parameters evaluated at the identical resource level are compared, and the candidate demonstrating top $1/\eta$ performance is “promoted”—allocated resources augmented by a factor of η and assigned to the available evaluator along with the corresponding model checkpoint. In the absence of promising candidates (e.g., in the beginning stage) or if all have previously been promoted, a new hyper-parameter configuration is suggested and evaluated from scratch with the initial resource.

The original ASHA implementation samples new hyper-parameters randomly from the search space, resulting in an unguided and inefficient search process.

In contrast, HyperRPS introduces the aforementioned multi-fidelity surrogate, complemented by time-constraint modeling, to suggest new hyper-parameters, significantly amplifying the efficacy of the multi-fidelity tuning algorithm.

Storage management. The growth of model sizes poses a significant challenge, particularly in the context of asynchronous scheduling methodologies. As model checkpoints are preserved for continuous training, the requirement for storage space ascends linearly with the duration of the search. This unbridled expansion in storage requisites, as revealed in our empirical investigation tuning the Mask2Former (Swin-L) with ASHA, culminates in the consumption of over 100 GB of storage capacity within a mere 24-hour timeframe. Such a scenario underscores the criticality of efficacious storage management strategies to preclude unforeseen computational impediments.

ASHA orchestrates a reservoir of hyper-parameter contenders for comparison and promotion. Nevertheless, given that solely the premier $1/\eta$ candidates are earmarked for continuous training, a significant proportion of the stored model checkpoints remain dormant until the end. Driven by this observation, HyperRPS develops a dynamic checkpoint retention algorithm to optimize storage occupancy. Concretely, for each early-stopping level i ($i = 0, 1, \dots, K-1$), we define the maximum number of model checkpoints (i.e., the storage threshold) as $S_{\text{max},i}$. If the storage of a resource level exceeds the threshold, all hyper-parameters within this level are ranked by their performance, and the model checkpoint of the worst candidate is then deleted. The hyper-parameter is marked as cleared, thereby precluding its future promotion by the scheduler, even in scenarios where it might ascend into the top $1/\eta$ bracket following the evaluation of additional hyper-parameters (albeit such an occurrence is of low probability).

The determination of $S_{\text{max},i}$ and the balance of this threshold across varying resource levels merit careful consideration. Within our multi-fidelity framework, the pool of candidates at each resource level necessitates sufficient samples for robust comparison and accurate promotion. To this end, a uniform $S_{\text{max},i}$ is instituted across each level i . We assign a global maximum number of model checkpoints (i.e., the global storage threshold) as S_{max} . The individual storage thresholds are derived as $S_{\text{max},i} = S_{\text{max}}/K$, with K representing the

total number of early-stopping levels. We will discuss the setting of the global storage threshold $S_{\max}$ in the experiment.

4.3.4 Tuning workflow

We provide pseudo-code for the proposed tuning algorithm in Algorithm 1. The optimizer runs asynchronously and loops until the search time budget is exhausted (Lines 2–7). It first checks if any evaluator returns an observation and then updates it in the history. If the number of saved model checkpoints exceeds the storage threshold, superfluous checkpoints are erased from the storage based on the storage management algorithm (Section 4.3.3). The corresponding hyper-parameters are marked as cleared and no longer resumable for continuous training. Once an evaluator is free, the `get_suggestion()` procedure is called, which returns the hyper-parameter and resource for the evaluator.

In the `get_suggestion()` procedure, the scheduler checks if a promising candidate has top performance in the history data (Lines 11–13). If one exists, the hyper-parameter is promoted and allocated with more resources to train (Section 4.3.3). Otherwise (Lines 15 and 16), a new hyper-parameter configuration is sampled based on the multi-fidelity surrogate with time constraint (Sections 4.3.1 and 4.3.2), and allocated with the initial resource.

Algorithm 1 Pseudo-code for the proposed algorithm

Input: Initial training resource r_{init} , maximum resource R , promote proportion η , search time budget T

```

1: Function run_HyperRPS(·)
2:   while current_time(·) <  $T$ 
3:     Collect observations and update history  $D$ ;
4:     Clear storage based on storage management algorithm;
5:     if an evaluator is free
6:        $x, r = \text{get\_suggestion}(\cdot)$ ;
7:       Assign hyper-parameter  $x$  and resource  $r$  to the evaluator;
8: Function get_suggestion(·)
9:   //  $r_c$ : Previously allocated resource for hyper-parameter  $c$ ;
10:  //  $D_{r_c}$ : The history with resource  $r_c$ ;
11:  if exists  $c \in \text{top } 1/\eta$  hyper-parameters in  $D_{r_c}$  and  $r_c < R$  and  $c$  is not promoted and not cleared
12:    Mark  $c$  as promoted;
13:    return  $c, r_c * \eta$ ;
14:  Sample new hyper-parameter  $x_{\text{new}}$  based on the optimizer;
15:  return  $x_{\text{new}}, r_{\text{init}}$ 

```

4.4 Framework implementation

To further speed up the hyper-parameter tuning procedure, algorithms are employed to evaluate multiple candidates concurrently. Notwithstanding, the burgeoning sizes of models, particularly in the context of optimizing Mask2Former, pose substantial challenges in accommodating multiple model instances within a single machine. This predicament necessitates the adoption of a distributed approach to tuning, leveraging multiple computational resources.

To address this challenge, HyperRPS is designed to facilitate collaborative tuning across a distributed computing environment by implementing a message queue communication structure. As illustrated in Fig. 4, messages are passed between the HyperRPS optimizer node and the evaluator nodes through two queues, the suggestion queue and the result queue. Initially, the optimizer node dispatches a suggestion encapsulating the hyper-parameter and the allocated resource budget into the suggestion queue. Subsequently, a free evaluator node retrieves the suggestion and trains the Mask2Former model. Upon completion of the training phase, the evaluator node reports the achieved performance metrics back to the result queue. The optimizer node, in turn, accesses these results from the result queue and updates its inner observation history. This architecture's inherent flexibility and modularity enable the straightforward deployment of evaluator nodes across disparate computational resources, thereby enhancing the scalability and efficiency of the hyper-parameter tuning process.

The basic Bayesian BO, including the surrogate model and acquisition function, are built based on OpenBox^[45, 46], while the advanced multi-fidelity tuning algorithm is implemented originally in HyperRPS.

5 Experiment

In this section, we present empirical results of HyperRPS compared with state-of-the-art methods and demonstrate the effectiveness of the proposed components in our framework.

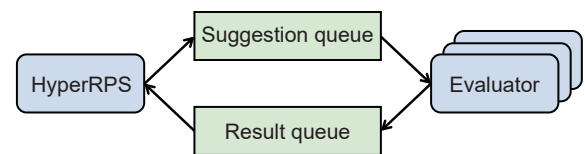


Fig. 4 Diagram of the distributed message queue structure.

5.1 Experiment setup

Datasets: We conduct the experiments on two datasets. As described in Section 3.1, the rock particle segmentation dataset contains 10 000 rock thin-section views labeled with mineral particle edges and category attributes. The training, validation, and test sets have 8000, 1000, and 1000 samples, respectively.

For reproducibility, we also conduct experiments on the public YouTube-VIS (2019)^[47] dataset. YouTube-VIS is an instance segmentation dataset that contains 2883 high-resolution YouTube videos. The category label set includes 40 common objects, such as persons, animals, and vehicles. Since only training labels are available for downloading, we split the original training set (2238 samples) into the training set, validation set, and test set as 8:1:1.

In hyper-parameter tuning, the Mask2Former model is trained on the training set. Tuning algorithms aim to optimize the performance score on the validation set. Finally, the tuning result is reported on the test set.

Metrics: AP is defined as the area under the precision-recall (PR) curve. The AP is averaged over ten intersection-over-union (IoU) thresholds from 50% to 95% at step 5%, following the evaluation protocol of the Common Objects in Context (COCO) benchmark. We also report the AP_{50} and AP_{75} in the experiments, which are computed at a single IoU of 50% and 75%, respectively.

Compared baselines: The compared baselines in the experiments are as follows: (1) Random search^[14]. Random search samples a random hyper-parameter configuration at each iteration, serving as a trivial approach. (2) EA^[17]. EA manages a hyper-parameter population, continuously evolving through selection, mutation, and crossover processes to generate viable new candidates. (3) ASHA^[23]. ASHA is an asynchronous multi-fidelity method. It samples new hyper-parameters randomly. (4) BOHB^[22]. BOHB combines the probabilistic modeling of BO with the efficient resource allocation strategy of Hyperband to balance exploration and exploitation. (5) BO^[46]. BO is a state-of-the-art baseline that utilizes a surrogate to model the search space and an acquisition function to balance exploration and exploitation.

Implementation details: The search time budget is 15 hours for tuning Mask2Former (Swin-B) and 24 hours for tuning Mask2Former (Swin-L). All experiments are repeated five times, and the mean and

standard deviation of the metrics are reported. All ablation studies are conducted by tuning Mask2Former (Swin-B) on the rock particle segmentation dataset. The experiments are conducted on Linux servers with two AMD EPYC 7763 64-core processors and eight Nvidia A800-80GB GPUs.

For parameters in HyperRPS, we set the maximum resource $R = 2000$ training iterations, the initial resource $r_{\text{init}} = 222$ training iterations, promotion proportion $\eta = 3$, and the number of early-stopping levels $K = 2$. The time-constraint threshold T_{max} is set to 1.1 times that of the training time of default hyper-parameter. The storage threshold $S_{\text{max}} = 18$ (25% of ASHA’s storage requirement).

5.2 Results on instance segmentation tasks

To demonstrate the efficiency of our proposed framework, we conduct empirical evaluations on both the public YouTube-VIS dataset and our rock particle segmentation dataset for tuning hyper-parameters of Mask2Former (Swin-B). To further assess the applicability of our framework in real-world production scenarios, we conduct a comparative analysis against the top three ranking methods, BO, as well as BOHB, focusing on the optimization of Mask2Former (Swin-L) on the rock particle segmentation dataset.

The outcomes of our experiments are detailed in Table 2. The methodologies of random search and ASHA exhibit minimal enhancements in model performance, attributed to their inherent nature of unguided search. Although BO demonstrates a marginal superiority over random search, its inability to adequately explore the search space within a limited search budget results in only nominal improvements. Since BOHB only utilizes full-fidelity observations to train the surrogate model, it may fail to gather sufficient data for effective modeling within a limited time budget, which can occasionally result in inferior performance compared with BO.

Conversely, our framework, with the utilization of the multi-fidelity surrogate and asynchronous early-stopping scheduling, consistently outperforms other methods across all evaluated metrics (AP_{50} , AP_{75} , and AP) for each task under consideration. Concretely, in comparison to the default hyper-parameter, HyperRPS facilitates significant improvements in the AP of Mask2Former (Swin-B), enhancing performance by 0.79 (+1.36%) and 2.69 (+8.26%) on the YouTube-VIS

Table 2 Results on instance segmentation tasks. Each entry reports “mean $\pm$ standard deviation” over five independent runs. Bold numbers indicate the best performance within each column. Symbol “/” denotes settings where a method was not evaluated; on Mask2Former (Swin-L), we only compare BO, BOHB, and HyperRPS, the top-3 performers on Mask2Former (Swin-B).

Method	YouTube-VIS (M2F Swin-B)			Rock particle (M2F Swin-B)			Rock particle (M2F Swin-L)		
	AP ₅₀	AP ₇₅	AP	AP ₅₀	AP ₇₅	AP	AP ₅₀	AP ₇₅	AP
Default	79.94 $\pm$ 0.32	65.93 $\pm$ 0.56	58.01 $\pm$ 0.24	57.42 $\pm$ 0.68	33.59 $\pm$ 0.54	32.58 $\pm$ 0.46	71.89 $\pm$ 0.24	43.83 $\pm$ 0.25	41.67 $\pm$ 0.19
Random	80.84 $\pm$ 0.22	67.35 $\pm$ 0.26	58.58 $\pm$ 0.12	58.94 $\pm$ 0.52	34.61 $\pm$ 0.75	33.55 $\pm$ 0.50	/	/	/
EA	79.76 $\pm$ 0.46	66.16 $\pm$ 0.50	58.37 $\pm$ 0.14	59.01 $\pm$ 0.62	34.64 $\pm$ 0.37	33.09 $\pm$ 0.25	/	/	/
BO	80.92 $\pm$ 0.18	66.16 $\pm$ 0.33	58.73 $\pm$ 0.18	59.38 $\pm$ 0.65	34.89 $\pm$ 0.46	33.76 $\pm$ 0.42	72.54 $\pm$ 0.10	44.64 $\pm$ 0.15	42.34 $\pm$ 0.11
ASHA	80.81 $\pm$ 0.33	66.28 $\pm$ 0.43	58.42 $\pm$ 0.39	58.55 $\pm$ 0.68	34.55 $\pm$ 0.73	33.42 $\pm$ 0.62	/	/	/
BOHB	81.01 $\pm$ 0.29	67.38 $\pm$ 0.41	58.69 $\pm$ 0.23	59.65 $\pm$ 0.59	34.72 $\pm$ 0.59	34.72 $\pm$ 0.67	72.81 $\pm$ 0.23	44.26 $\pm$ 0.13	42.29 $\pm$ 0.27
HyperRPS	81.96 $\pm$ 0.32	67.73 $\pm$ 0.40	58.80 $\pm$ 0.16	61.01 $\pm$ 0.67	35.77 $\pm$ 0.65	35.27 $\pm$ 0.52	73.12 $\pm$ 0.17	45.10 $\pm$ 0.18	42.83 $\pm$ 0.20

and rock particle segmentation datasets, respectively. Moreover, it improves the AP of Mask2Former (Swin-L) by 1.16 (+2.78%). Notably, the gains in performance are more significant on the rock particle segmentation dataset than on YouTube-VIS, underscoring the necessity of hyper-parameter tuning for domain-specific applications.

Our system not only significantly improves the AP but also incurs minimal overhead: across all experiments, the average time required to recommend a new configuration using the multi-fidelity surrogate model is 4.7 s, with a similar time cost for time-constraint modeling. This time is negligible as the modeling process can run in parallel with the training of Mask2Former.

5.3 Parameter importance analysis

We delve into the empirical results through a comprehensive analysis of the importance attributed to various hyper-parameters. For this purpose, we employ the FANOVA^[48] framework, which fits a random forest model with tuning history and outputs the importance score of each hyper-parameter. As delineated in Table 3, the analysis reveals that the optimizer-related hyper-parameters, notably the optimizer type, momentum, learning rate, and backbone multiplier, emerge as the most critical. Among the data augmentation hyper-parameters, saturation stands out as particularly influential. These findings are poised to contribute valuable insights to the discourse on hyper-parameter tuning within the research community.

5.4 Search space comparison

To further investigate the efficacy of our designed

Table 3 Parameter importance analysis.

Hyper-parameter	Importance score
Optimizer type	0.5191
Momentum	0.2039
Learning rate	0.1544
Backbone multiplier	0.0651
Saturation	0.0072
Weight decay	0.0051
Brightness	0.0003
Clip gradients value	0.0003
Contrast	0.0001

search space, we conduct an ablation study comparing search spaces of varying sizes: Small, medium, and full. The small space consists of only two hyper-parameters: Learning rate and weight decay. The medium space includes all five optimizer-related hyper-parameters listed in Table 1, along with the clip gradients value hyper-parameter. On the other hand, the full space is the proposed space comprising nine hyper-parameters.

As depicted in Fig. 5, our result shows that tuning

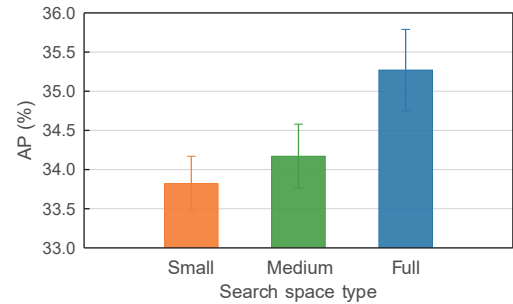


Fig. 5 Ablation study on search space. Bars show the mean AP, and the T-shaped error bars indicate the standard deviation.

hyper-parameters in the small space results in sub-optimal performance. Tuning in the medium space slightly improves the final AP of the model by 0.35. However, the best AP is achieved while tuning in the full space with a significant margin of 1.45 AP compared with the small space. This highlights the efficacy of our proposed search space design that incorporates both optimizer-related and data-augmentation hyper-parameters.

5.5 Effectiveness of time-constraint modeling

We conduct an ablation study on the time-constraint modeling technique in HyperRPS. As shown in Table 4, tuning with time-constraint modeling leads to a reduction of 7.41% on constraint violation probability, thus saving search time from time-consuming stragglers for space exploration while enhancing the final AP by an average of 0.58.

5.6 Scalability with different dataset size

We evaluate the scalability of HyperRPS through expanding the size of the rock particle segmentation dataset from 10 000 to 25 000. Table 5 shows the percentage improvement of HyperRPS compared with the default configuration in different sizes. As the dataset size increases from 10k to 15k, HyperRPS yields substantial performance improvements. However, beyond 20k, the improvements diminish due to two factors: (1) Default configurations improve with

Table 4 Ablation study on time-constraint modeling. Δ denotes the difference between with (w/) time constraint and without (w/o) time constraint.

Setting	Violation probability	AP
w/o time constraint	15.74 ± 1.73	34.69 ± 0.57
w/ time constraint	8.33 ± 1.96	35.27 ± 0.52
Δ	-7.41	+0.58

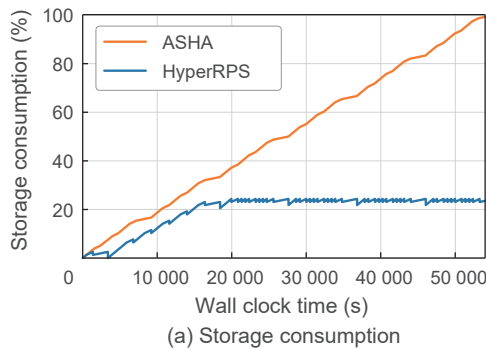


Table 5 Improvement of optimization compared with the default configuration with dataset scaling across different sizes.

Dataset size	Improvement of AP (%)
10 000	8.26
15 000	10.08
20 000	7.97
25 000	6.65

larger datasets, and (2) fewer hyperparameter configurations can be evaluated within a fixed budget, limiting the surrogate model's fitting ability. Despite this, HyperRPS consistently outperforms the default configuration by a notable margin.

5.7 Storage reduction

We conduct a comparison of HyperRPS with ASHA in terms of storage consumption. According to Fig. 6, ASHA exhibits uncontrolled storage behavior, leading to linearly increasing storage consumption with search time. In contrast, HyperRPS employs a storage management algorithm to restrict storage under a threshold effectively. This results in a maximum storage requirement reduction of over 75% compared with ASHA.

To determine the appropriate storage threshold for HyperRPS, we conduct experiments with threshold values of 3%, 10%, 25%, 50%, and 100% of ASHA's maximum storage requirement. As shown in Fig. 7, setting a low threshold value, such as 3% or 10%, results in an insufficient hyper-parameter candidate pool for accurate promotion, rendering the algorithm equivalent to random search. On the other hand, the 25% storage threshold does not compromise performance while reducing storage space compared with 100% storage. Consequently, we choose 25% ($S_{\max} = 18$) as the storage threshold for HyperRPS.

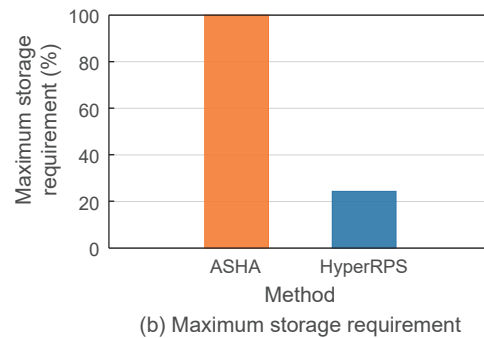


Fig. 6 Storage comparison with ASHA.

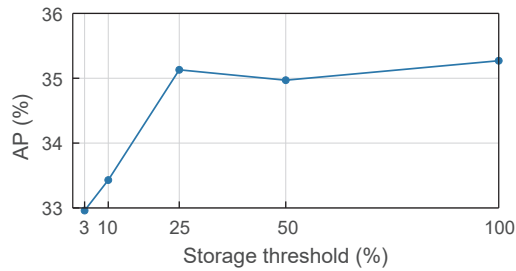


Fig. 7 Effects of different storage thresholds.

6 Conclusion

In this paper, we present HyperRPS, an efficient multi-fidelity hyperparameter tuning framework for rock particle segmentation. HyperRPS elaborately designs a search space and proposes a multi-fidelity hyperparameter tuning algorithm to speed up the tuning process with three techniques, including (1) multi-fidelity surrogate, (2) time-constraint modeling, and (3) asynchronous scheduling with storage management. The empirical results on both the public segmentation dataset and the rock particle segmentation dataset demonstrate its superior performance against state-of-the-art tuning algorithms.

Future work includes further expanding the search space to encompass more deep learning hyperparameters, such as network width and depth, and introducing network architecture search methods to increase the breadth and flexibility of the search. Another promising method to further improve performance is ensembling. We can fully leverage the validated hyper-parameters explored during the optimization process by selecting diverse and high-performing ones to train models and ensemble them. Thus, the final generalization capability and predictive performance can be further enhanced.

Acknowledgment

This work was supported by the National Natural Science Foundation of China (No. 42372175), National Science and Technology Major Project (No. 2025ZD1401501), the Foundation for Outstanding Research Groups of the National Natural Science Foundation of China (No. 72088101), the China National Petroleum Corporation–Peking University Strategic Cooperation Project of Fundamental Research, and the China National Petroleum Corporation Technology Project (No. 2023DJ84).

Declaration of competing interest

The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] H. Liu, Y. L. Ren, X. Li, Y. X. Hu, J. P. Wu, B. Li, L. Luo, Z. Tao, X. Liu, J. Liang, et al., Rock thin-section analysis and identification based on artificial intelligent technique, *Pet. Sci.*, vol. 19, no. 4, pp. 1605–1621, 2022.
- [2] Y. Ren, X. Li, J. Bi, Y. Zhang, Q. Su, W. Wang, and H. Li, Multi-channel attention transformer for rock thin-section image segmentation, *J. Eng. Res.*, vol. 13, no. 2, pp. 730–740, 2025.
- [3] Z. Y. Zhang, Research on image segmentation and recognition of sandstone thin section, (in Chinese), Master dissertation, University of Science and Technology of China, Hefei, China, 2020.
- [4] Y. Zhong, S. H. Xiong, Q. Z. Teng, Y. K. Wu, and J. Gong, Particle segmentation method based on orthogonal polarization fusion image of rock slice, (in Chinese), *Cyber Secur. Data Gov.*, vol. 38, no. 10, pp. 87–92, 2019.
- [5] F. Jiang, Research on segmentation of sandstone thin section images based on semantic identification, (in Chinese), Ph.D. dissertation, Nanjing University, Nanjing, China, 2018.
- [6] G. Q. Wang, Z. Y. Wang, and D. S. Luo, Image segmentation of overlapping rock particles based on FCM and labeling watershed, (in Chinese), *J. Sichuan Univ. (Nat. Sci. Ed.)*, vol. 49, no. 2, pp. 356–360, 2012.
- [7] Z. Li, Q. Z. Teng, and Y. Q. Zhang, Granule segmentation of orthogonal polarization sequence diagrams of rock thin section, (in Chinese), *SmartTech Innov.*, vol. 3, pp. 26–32, 2018.
- [8] M. Y. Fang, J. N. He, W. H. Wang, and M. Z. Li, Research on a method of particle size analysis of clastic rock based on MATLAB, (in Chinese), *Acta Petrol. Mineral.*, vol. 39, no. 4, pp. 504–510, 2020.
- [9] Y. Q. Jiang, J. X. Zhou, J. X. Feng, and Q. Z. Teng, The application of DBSCAN algorithm and mathematical morphology in rock slice image segmentation, (in Chinese), *Cyber Secur. Data Gov.*, vol. 35, no. 17, pp. 39–41&48, 2016.
- [10] C. R. Si, R. C. Wang, K. Di, and P. G. Zhu, A rock particle image segmentation method based on mask R-CNN and watershed algorithm, (in Chinese), *Water Resour. Power*, vol. 38, no. 11, pp. 129–132&128, 2020.
- [11] S. Pattnaik, S. Chen, A. Helba, and S. Ma, Automatic carbonate rock facies identification with deep learning, in *Proc. SPE Annu. Technical Conf. and Exhibition*, Virtual Event, 2020, p. SPE-201673-MS.
- [12] K. He, G. Gkioxari, P. Dollár, and R. Girshick, Mask R-CNN, in *Proc. IEEE Int. Conf. Computer Vision*, Venice, Italy, 2017, pp. 2980–2988.
- [13] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R.

- Girdhar, Masked-attention mask transformer for universal image segmentation, in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, New Orleans, LA, USA, 2022, pp. 1280–1289.
- [14] E. Austin, S. Makwana, A. Trabelsi, C. Langeron, and O. R. Zaïane, Uncovering flat and hierarchical topics by community discovery on word co-occurrence network, *Data Sci. Eng.*, vol. 9, no. 1, pp. 41–61, 2024.
- [15] J. Bergstra and Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.
- [16] T. Bäck and H. P. Schwefel, An overview of evolutionary algorithms for parameter optimization, *Evol. Comput.*, vol. 1, no. 1, pp. 1–23, 1993.
- [17] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, Regularized evolution for image classifier architecture search, in *Proc. 33rd AAAI Conf. Artificial Intelligence*, Honolulu, HI, USA, 2019, pp. 4780–4789.
- [18] F. Hutter, H. H. Hoos, and K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, in *Proc. 5th Int. Conf. Learning and Intelligent Optimization*, Rome, Italy, 2011, pp. 507–523.
- [19] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, Algorithms for hyper-parameter optimization, in *Proc. 25th Int. Conf. Neural Information Processing Systems*, Granada, Spain, 2011, pp. 2546–2554.
- [20] Y. Li, G. Zang, C. Song, X. Yuan, and T. Ge, Leveraging semantic information for enhanced community search in heterogeneous graphs, *Data Sci. Eng.*, vol. 9, no. 2, pp. 220–237, 2024.
- [21] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, Hyperband: Bandit-based configuration evaluation for hyperparameter optimization, in *Proc. 5th Int. Conf. Learning Representations*, Toulon, France, 2017, pp. 1–48.
- [22] S. Falkner, A. Klein, and F. Hutter, BOHB: Robust and efficient hyperparameter optimization at scale, in *Proc. 35th Int. Conf. Machine Learning*, Stockholm, Sweden, 2018, pp. 1436–1445.
- [23] L. Li, K. G. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-Tzur, M. Hardt, B. Recht, and A. Talwalkar, A system for massively parallel hyperparameter tuning, in *Proc. 3rd Conf. Machine Learning and Systems*, Austin, TX, USA, 2020, pp. 230–246.
- [24] Y. Li, Y. Shen, H. Jiang, W. Zhang, J. Li, J. Liu, C. Zhang, and B. Cui, Hyper-tune: Towards efficient hyper-parameter tuning at scale, *Proc. VLDB Endow*, vol. 15, no. 6, pp. 1256–1265, 2022.
- [25] R. Samet, Ş. E. Amrahov, and A. H. Ziroğlu, Fuzzy rule-based image segmentation technique for rock thin section images, in *Proc. 3rd Int. Conf. Image Processing Theory, Tools and Applications*, Istanbul, Turkey, 2012, pp. 402–406.
- [26] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining*, Portland, OR, USA, 1996, pp. 226–231.
- [27] X. Q. Hu, S. C. Gao, X. Zhang, W. Wang, H. R. Zhong, and H. Zhou, A rock thin section analysis system based on continuous extinction characteristics, (in Chinese), *Earth Sci. Front.*, vol. 28, no. 2, pp. 296–304, 2021.
- [28] Y. Wu, G. F. Su, Q. Z. Teng, X. H. He, and L. Guo, A particles segmentation method of rock slice orthogonal polarization images, (in Chinese), *Sci. Technol. Eng.*, vol. 13, no. 31, pp. 9201–9206, 2013.
- [29] H. Hu, X. Yue, H. Q. Bai, W. Y. Li, J. P. Li, and W. Hong, Application of intelligent image algorithm based on super-pixel in mineral particle segmentation, (in Chinese), *China Offshore Oil Gas*, vol. 33, no. 2, pp. 89–95, 2021.
- [30] H. Zhou, C. L. Zhang, X. Zhang, Q. X. Chen, Y. Zhang, and C. C. Zhong, Edge extraction and particle segmentation based on coherent features of rock slice sequence images, (in Chinese), *J. Jilin Univ. (Earth Sci. Ed.)*, vol. 51, no. 6, pp. 1897–1907, 2021.
- [31] O. Ronneberger, P. Fischer, and T. Brox, U-Net: Convolutional networks for biomedical image segmentation, in *Proc. 18th Int. Conf. Medical Image Computing and Computer-Assisted Intervention*, Munich, Germany, 2015, pp. 234–241.
- [32] F. Hutter, L. Kotthoff, and J. Vanschoren, *Automated Machine Learning: Methods, Systems, Challenges*. Cham, Switzerland: Springer, 2019.
- [33] S. Huang, Y. Qin, X. Zhang, Y. Tu, Z. Li, and B. Cui, Survey on performance optimization for database systems, *Sci. China Inf. Sci.*, vol. 66, no. 2, p. 121102, 2023.
- [34] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. New York, NY, USA: John Wiley & Sons, 2017.
- [35] K. G. Jamieson and A. Talwalkar, Non-stochastic best arm identification and hyperparameter optimization, in *Proc. 19th Int. Conf. Artificial Intelligence and Statistics*, Cadiz, Spain, 2016, pp. 240–248.
- [36] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, Swin transformer: Hierarchical vision transformer using shifted windows, in *Proc. IEEE/CVF Int. Conf. Computer Vision*, Montreal, Canada, 2021, pp. 9992–10002.
- [37] C. E. Rasmussen, Gaussian processes in machine learning, in *Proc. Advanced Lectures on Machine Learning*, Canberra, Australia, 2004, pp. 63–71.
- [38] W. M. Hu, Q. Wang, J. Gao, B. Li, and S. Maybank, DCFNet: Discriminant correlation filters network for visual tracking, *J. Comput. Sci. Technol.*, vol. 39, no. 3, pp. 691–714, 2024.
- [39] I. Sutskever, J. Martens, G. E. Dahl, and G. E. Hinton, On the importance of initialization and momentum in deep learning, in *Proc. 30th Int. Conf. Machine Learning*, Atlanta, GA, USA, 2013, pp. 1139–1147.
- [40] S. Y. Zhao, C. W. Shi, Y. P. Xie, and W. J. Li, Stochastic normalized gradient descent with momentum for large-batch training, *Sci. China Inf. Sci.*, vol. 67, no. 11, p. 212101, 2024.

- [41] Y. Wu, A. Kirillov, F. Massa, W. Y. Lo, and R. Girshick, Detectron2, <https://github.com/facebookresearch/detectron2>, 2019.
- [42] H. C. Liu, L. F. Dong, and X. M. Zhang, Multimodal dependence attention and large-scale data based offline handwritten formula recognition, *J. Comput. Sci. Technol.*, vol. 39, no. 3, pp. 654–670, 2024.
- [43] D. R. Jones, M. Schonlau, and W. J. Welch, Efficient global optimization of expensive black-box functions, *J. Global Optim.*, vol. 13, no. 4, pp. 455–492, 1998.
- [44] B. Letham, B. Karrer, G. Ottoni, and E. Bakshy, Constrained Bayesian optimization with noisy experiments, *Bayesian Anal.*, vol. 14, pp. 495–519, 2019.
- [45] Y. Li, Y. Shen, W. Zhang, Y. Chen, H. Jiang, M. Liu, J. Jiang, J. Gao, W. Wu, Z. Yang, et al., OpenBox: A generalized black-box optimization service, in *Proc. 27th ACM SIGKDD Conf. Knowledge Discovery & Data Mining*, Singapore, 2021, pp. 3209–3219.
- [46] H. Jiang, Y. Shen, Y. Li, B. Xu, S. Du, W. Zhang, C. Zhang, and B. Cui, OpenBox: A python toolkit for generalized black-box optimization, *J. Mach. Learn. Res.*, vol. 25, no. 1, p. 120, 2024.
- [47] L. Yang, Y. Fan, and N. Xu, Video instance segmentation, in *Proc. IEEE/CVF Int. Conf. Computer Vision*, Seoul, Republic of Korea, 2019, pp. 5187–5196.
- [48] F. Hutter, H. Hoos, and K. Leyton-Brown, An efficient approach for assessing hyperparameter importance, in *Proc. 31st Int. Conf. Machine Learning*, Beijing, China, 2014, pp. 754–762.



Yili Ren is a senior engineer at the Research Institute of Petroleum Exploration & Development (RIPED), PetroChina, China; the Artificial Intelligence Technology R&D Center of Exploration and Development, China National Petroleum Corporation, China; and the National Key Laboratory for

Multi-Resources Collaborative Green Production of Continental Shale Oil, China. She is currently a postdoctoral researcher at Department of Automation, Tsinghua University, China. She received the master degree in 2011 from Beihang University, China, and the PhD degree in 2023 from the RIPED, PetroChina, China. Her research interests include application of computer vision, deep learning, and large model in oil and gas geology.



Beicheng Xu is a master student at School of Computer Science Peking University, China. He received the BEng degree in computer science and technology from Wuhan University, China, in 2023. His research interests include hyper-parameter optimization, black-box optimization, and automated machine learning.



Huaijun Jiang is a master student at Key Laboratory of High Confidence Software Technologies (MOE), School of Computer Science, Peking University, China. He received the BS degree in computer science and technology from Peking University, China, in 2021. His research interests include hyperparameter

optimization, black-box optimization, and automatic machine learning.



Xi Liu is an algorithm engineer at the Research Institute of Petroleum Exploration & Development, PetroChina, China, and the Artificial Intelligence Technology R&D Center of Exploration and Development, China National Petroleum Corporation, China. She received the master degree from Beijing

Technology and Business University, China, in 2021. Her research interests mainly focuses on application of deep learning in reservoir evaluation.



Ping Zhang is a senior engineer at Digital and Intelligent Management Department, PetroChina Changqing Oilfield Company, China. She received the master degree from Xidian University, China, in 2007. Her main research interests focus on digitalization of geological engineering.



Qian Liang is a senior engineer at Digital and Intelligent Management Department, PetroChina Changqing Oilfield Company, China. She received the master degree in 2007 from Northwestern Polytechnical University, China. Her main research direction is the application of big data analysis and artificial intelligence.



Bin Cui is a professor and the vice dean of School of Computer Science, Peking University, China. He has served for the Technical program committees of various international conferences including SIGMOD, VLDB, ICDE, and KDD, and is an editor-in-chief of *Data Science and Engineering*. He is serving as the vice chair of Technical Committee on Database CCF (2020–2027). He is a member of IEEE. His research interests include database system, AI system and application, and big data management and analytics.



Yu Shen is a PhD student at School of Computer Science at Peking University, China. His research interests include configuration optimization, automated machine learning, and their practical applications.